Person 2: `drone.py` (the authentication protocol, M1 → M2 → M3)

Your job in one line: explain the 3-message handshake: how two drones that have never met prove who they are and agree on a fresh secret key over a radio channel the attacker controls.

Run your demo: `python drone.py`

---

1. What to say (about 3 minutes)

Draw this on the board first:

```
Drone A                                              Drone B
   |  M1: cert_A, ephA, nA, tsA, Sig_A(M1)             |
   | ------------------------------------------------> |  B checks: to-me? fresh? new nonce?
   |                                                   |            cert by GCS? signature?
   |  M2: cert_B, ephB, nB, tsB, Sig_B(M1 || M2)       |
   | <------------------------------------------------ |
   |  A checks the same 5 things                       |
   |  both compute  shared = DH(ephA, ephB)            |
   |  M3: HMAC(K_mac, "A-confirm" || M1 || M2)         |
   | ------------------------------------------------> |  B checks the tag -> session ready
```

> "Each drone sends its **certificate** (who I am), a **fresh Diffie-Hellman public key** (to build a secret), a **random nonce** and a **timestamp** (so the message can't be replayed later), and **signs all of it** with its private key.
>
> The receiver checks five things: the message is addressed to it, the timestamp is fresh, the nonce is new, the certificate was signed by the GCS, and the signature matches the key in that certificate.
>
> B's signature covers **M1 and M2 together**, so B's reply is tied to *this* request and can't be cut and pasted into another run.
>
> Both drones then compute the same secret from Diffie-Hellman, which the attacker can't compute even after seeing both public keys. Finally, A sends **M3**, an HMAC that proves 'I derived the same key as you'. This is the same idea as the TLS handshake, adapted for drones without a server."

---

2. Libraries in this file

Import	Where it's from	Why we need it
`os`	stdlib	`os.urandom(16)` : cryptographically secure random nonces

Import	Where it's from	Why we need it
`time`	stdlib	Timestamps (freshness check)
`Ed25519PrivateKey`	`cryptography`	The drone's long-term signing key
`X25519PrivateKey`, `X25519PublicKey`	`cryptography`	Ephemeral elliptic-curve Diffie-Hellman
`from gcs import ...`	our file (Person 1)	Registration, certificate check, signature check, `ser`, `pub_hex`, `AuthError`
`from session import ...`	our file (Person 3)	Key derivation, confirmation tag, the encrypted `Session`

---

3. Line by line

```
MAX_SKEW = 30
```

A timestamp more than 30 seconds away from my clock means "too old, probably replayed". Drones have GPS, and GPS gives accurate time, so a 30 s window is realistic.

```
def dh(my_eph, peer_eph_hex):
    return my_eph.exchange(X25519PublicKey.from_public_bytes(bytes.fromhex(peer_eph_hex)))
```

Diffie-Hellman. A has secret `a` and sends `aG`; B has secret `b` and sends `bG`. A computes `a·(bG)`, B computes `b·(aG)`, and both get `abG`. An eavesdropper sees `aG` and `bG` but can't get `abG`. That's the **CDH assumption**. X25519 is DH on Curve25519.

`Drone.__init__`, setting up a drone

```
self.id = drone_id
self._sk = Ed25519PrivateKey.generate()      # long-term signing key, never leaves the drone
self.gcs_pk = gcs.pk                         # trust anchor
self.crl = gcs.revoked                       # revocation list
self.cert = gcs.register(drone_id, pub_hex(self._sk))
self._seen_nonces = set()                    # replay cache
self._pending = {}                           # half-finished handshakes
```

- The drone **makes its own key**. Only the public half goes to the GCS in `register(...)`.
- It stores the GCS public key so it can check other drones' certificates offline.
- `_seen_nonces` remembers every nonce already accepted, so a replayed message is spotted.
- `_pending` stores state between messages, for example A must remember its secret `eph` from M1 until M2 arrives.

`_unsigned`, what exactly gets signed

```
return ser({k: v for k, v in msg.items() if k != "sig"})
```

You can't sign the signature itself, so we sign every field *except* `sig`, using the canonical `ser` from `gcs.py`.

`_check`, the five security checks on M1/M2

```
if msg.get("type") != expected_type or msg.get("to") != self.id:   # 1. right message, meant for me
if abs(time.time() - msg["ts"]) > MAX_SKEW:                        # 2. fresh (timestamp)
if msg["n"] in self._seen_nonces:                                  # 3. not a replay (nonce)
pk = verify_certificate(msg["cert"], self.gcs_pk, self.crl, msg["from"])   # 4. legit drone (gcs.py)
if not verify_sig(pk, msg["sig"], signed):                         # 5. sender HOLDS the private key
self._seen_nonces.add(msg["n"])                                    # remember the nonce only if all
passed
```

Check 4 proves "the GCS vouched for this public key". Check 5 proves "the sender owns that key". **You need both.** A copied certificate passes check 4 but fails check 5. The nonce is recorded *after* all checks pass, so junk messages can't fill the cache with fake nonces.

`initiate`, A builds M1

```
eph = X25519PrivateKey.generate()            # fresh DH key, used for one session only
m1 = {"type": 1, "from": self.id, "to": peer_id, "cert": self.cert,
      "eph": pub_hex(eph), "n": os.urandom(16).hex(), "ts": time.time()}
m1["sig"] = self._sk.sign(self._unsigned(m1)).hex()
self._pending[peer_id] = (eph, m1)
```

A **fresh** DH key every session gives **forward secrecy**. The nonce is 16 random bytes (128 bits, so it never repeats by chance). A signs everything, so the attacker can't swap the DH key (the MITM attack test). A keeps `eph` (secret) and `m1` for later.

`respond`, B checks M1 and builds M2

```
self._check(m1, 1, self._unsigned(m1))
eph = X25519PrivateKey.generate()
m2 = {...same fields, from B to A...}
m2["sig"] = self._sk.sign(ser(m1) + self._unsigned(m2)).hex()
self._pending[m1["from"]] = (eph, m1, m2)
```

The key line is **`ser(m1) + self._unsigned(m2)`**. B signs A's message *together with* its own. This **binds** the reply to this exact request, including A's DH key and nonce. An attacker can't reuse an old M2, and can't change M1 without B's signature failing at A.

`finish`, A checks M2 and sends M3

```
eph, m1 = self._pending.pop(m2["from"])
self._check(m2, 2, ser(m1) + self._unsigned(m2))
k_enc, k_mac, transcript = derive_keys(dh(eph, m2["eph"]), m1, m2)
m3 = {"type": 3, "from": self.id, "to": m2["from"], "tag": make_tag(k_mac, transcript)}
return m3, Session(k_enc, 0, m2["from"])
```

A rebuilds exactly what B should have signed (M1 + M2) and checks it. Then: DH secret → `derive_keys` (in `session.py`) → two keys. The M3 tag proves to B that A really derived the same key. A's channel object is `Session(k_enc, role=0)`, where 0 means initiator.

`confirm`, B checks M3

```
eph, m1, m2 = self._pending.pop(m3["from"])
k_enc, k_mac, transcript = derive_keys(dh(eph, m1["eph"]), m1, m2)
if not check_tag(k_mac, transcript, m3["tag"]):
    raise AuthError("key confirmation failed")
return Session(k_enc, 1, m3["from"])
```

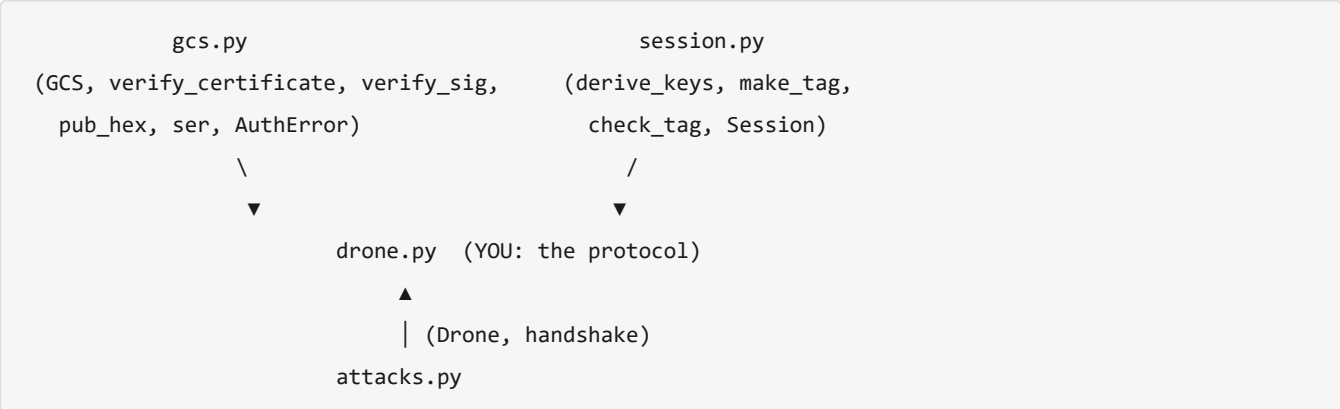
B computes the same keys from its side of the DH. A matching tag means A is alive, holds `ephA`, and saw the same M1 and M2. Only then does B open the channel (role 1 = responder).

`handshake`

Runs M1 → M2 → M3 for two honest drones in one call. `attacks.py` uses it for the honest baseline.

---

4. How this file links to the others



`drone.py` is the **centre**. It uses Person 1's identity checks and Person 3's key and channel code, and Person 4 attacks it.

---

5. Lecture-plan topics (CS428) and how to say them

Lecture topic	Where	How to explain it
Secure Authenticated Key exchange protocols	the whole M1/M2/M3 flow	"Authenticated DH: DH gives a shared secret, signatures make it *authenticated* so no MITM. It's close to the SIGMA / STS family used in TLS 1.3 and IKE."
CDH/DDH, **Discrete logarithm problem**	`dh()`	"Seeing aG and bG doesn't let you compute abG (CDH on an elliptic curve)."
Digital Signature Schemes	`sign` in `initiate` / `respond`, `verify_sig` in `_check`	"Signatures bind the DH keys to the drones' identities."
SSL / TLS, session and connection establishment between client & server	the 3 messages	"Same shape as the TLS handshake: hello + key share + certificate + signature + finished. Here both sides are peers, so authentication is mutual."
Cipher suites	our choice	"Our suite: X25519 key exchange, Ed25519 signatures, HKDF-SHA256, AES-256-GCM. It's comparable to TLS 1.3's."
Message Authentication Using Hash Functions	M3 HMAC tag	"Key confirmation is an HMAC over the transcript."
Application area: Vehicular Network	drones = flying vehicular network	"V2V and D2D have the same problems: fast-moving nodes, no server, open radio."

6. Likely questions

- **Why both signatures and an HMAC?** Signatures prove *identity* and protect the DH keys. M3 proves A actually *finished*: it holds the DH secret and computed the same key. Without M3, B can't be sure A is alive and agrees.
- **Why nonces AND timestamps?** A timestamp alone allows replay inside the 30 s window. Nonces alone need unbounded memory. Together, the timestamp rejects anything old and the nonce cache only needs to cover the last 30 s.
- **Why a fresh DH key every session (ephemeral)?** It gives forward secrecy. If a drone is captured tomorrow, its *signing* key doesn't reveal yesterday's session keys, because the `eph` secrets were deleted.
- **How is MITM stopped?** The attacker can swap `eph` for its own, but it can't re-sign the message without the drone's private key, so `verify_sig` fails. The attack tests `test_mitm_swaps_dh_key` and `test_mitm_on_response` show this.
- **Why does B sign M1 too?** To tie the response to the request. Otherwise an attacker could pair an old M2 with a new M1.
- **What are the honest limitations?** Only one pending handshake per peer at a time, no protection against flooding (signature checks cost CPU), and the CRL is simulated as a shared set.